

# Řetězce, soubory, rekurze III

IB111 ZÁKLADY PROGRAMOVÁNÍ

---

Tomáš Vojnar (na základě slajdů Nikoly Beneše)

28. listopadu 2025

# Řetězce

---

Jak reprezentovat text v počítači?

Co je to vlastně „text“?

Jak reprezentovat text v počítači?

Co je to vlastně „text“?

- Posloupnost znaků.

Jak reprezentovat text v počítači?

Co je to vlastně „text“?

- Posloupnost znaků.

Co je to „znak“?

## Trocha historie (zjednodušeně)

- První způsoby kódování textu:
  - námořní vlajky, semafore, Braillovo písmo, Morseovka, ...
- Baudot (cca 1870) – kódování pro telegraf:
  - 5 bitů (32 možností) stačí na všechno.
- **ASCII** (1963):
  - 7 bitů (128 možností) stačí na všechno
- 80. léta – různé „**kódové stránky**“:
  - 8 bitů (256 možností) stačí pro jednu „oblast“.
  - Prvních 128 možností je shodných s ASCII.
  - Dalších 128 má každá stránka jinak.
  - V danou chvíli je možné použít jen jedno kódování.
- Mezitím v Asii – dvoubajtová kódování:
  - spousta různých, vzájemně nekompatibilních.

## Trocha historie

- 90. léta – **Unicode**:
  - jeden univerzální standard zahrnující všechny potřebné znaky.
  - Oddělení tzv. kódových bodů od kódových jednotek.
  - Různá konkrétní kódování (populární UTF-8).
  - Postupně se rozrůstá (emoji apod.).
  - Reálné rozšíření na webu začíná cca 2006

Co je to „znak“?



Co je to „znak“? Základní jednotka textu, ale lze chápat různě...

- *Grafém* (*grapheme*):
  - Nejmenší funkční jednotka písma z pohledu čtenáře – nelze rozdělit bez ztráty významu (např. „š“).

Co je to „znak“? Základní jednotka textu, ale lze chápat různě...

- *Grafém* (*grapheme*):
  - Nejmenší funkční jednotka písma z pohledu čtenáře – nelze rozdělit bez ztráty významu (např. „š“).
- *Kódový bod* (*code point*) – atomický prvek textu v kódu.
  - Unicode: 1 114 112 možností, něco přes čtvrtinu obsazených.
  - Jeden grafém může být složen z více kódových bodů (viz např. *combining diacritical marks*, *emoji modifiers*).
  - Např. „š“ lze jako samostatný bod nebo jako složení „ ˇ “ a „s“.  
(Existuje samostatný a kombinující háček!)
  - Některé kódové body nejsou součástí grafémů (viz např. *zero-width joiner*).

Co je to „znak“? Základní jednotka textu, ale lze chápat různě...

- *Grafém* (*grapheme*):
  - Nejmenší funkční jednotka písma z pohledu čtenáře – nelze rozdělit bez ztráty významu (např. „š“).
- *Kódový bod* (*code point*) – atomický prvek textu v kódu.
  - Unicode: 1 114 112 možností, něco přes čtvrtinu obsazených.
  - Jeden grafém může být složen z více kódových bodů (viz např. *combining diacritical marks*, *emoji modifiers*).
  - Např. „š“ lze jako samostatný bod nebo jako složení „<sup>ˇ</sup>“ a „s“ (Existuje samostatný a kombinující háček!).
  - Některé kódové body nejsou součástí grafémů (viz např. *zero-width joiner*).
- *Kódová jednotka* (*code unit*):
  - jednotka v konkrétním kódování (do sekvence bitů).
  - UTF-8: 8 bitů, UTF-16: 16 bitů, UTF-32: 32 bitů.
  - Jeden kódový bod může být složen z více kódových jednotek.

## Řetězce v Pythonu

- Python umí pracovat s různými kódováními.
- Zdrojový soubor programu je implicitně v UTF-8 (Python 3).
- Interní reprezentace řetězců
  - komplikovanější (PEP-393), ale pokrývá celý Unicode.
- „Znaky“ jsou **kódové body** (code points).

## Řetězce v Pythonu

- Python umí pracovat s různými kódováními.
- Zdrojový soubor programu je implicitně v UTF-8 (Python 3).
- Interní reprezentace řetězců
  - komplikovanější (PEP-393), ale pokrývá celý Unicode.
- „Znaky“ jsou **kódové body** (code points).

## Syntax

- Uvozovky nebo apostrofy.
- Víceřádkové řetězce pomocí ztrojení uvozovek/apostrofů.
- Speciální znaky uvozené zpětným lomítkem:
  - konec řádku `\n`, tabulátor `\t`, uvozovka `\"`, apostrof `\'`,
  - samotné zpětné lomítko `\\`,
  - ... a různé jiné.

## Přístup k jednotlivým „znakům“ (kódovým bodům)

- **Indexování** `text[i]`.
  - Pozor: Python nemá datový typ pro *znak*, výsledkem je *jednoznakový řetězec*.
  - Jen pro čtení, řetězce jsou v Pythonu *neměnné*.
- **Délka** řetězce `len(text)` – počet kódových bodů.
- **Iterace** pomocí `for c in text:`.

## Vytváření nových řetězců

- **Zřetězení**: `"ahoj, " + "lidi"`.
  - `text += s` je totéž, co `text = text + s`.
- **Konverze** na velká/malá písmena: `.upper()`, `.lower()`.
- **Nahrazení podřetězce** náhradou: `text.replace(sub, repl)`.
- **Odstranění bílých znaků** z konce: `text.rstrip()`
  - mezery, tabulátory, konce řádků, ...

## Zjišťování vlastností řetězců (predikáty)

- `.islower()` – neprázdný řetězec splňující:
  - obsahuje alespoň jedno písmeno,
  - všechna písmena v řetězci jsou malá.
- `.isupper()` – podobně jako výše, všechna velká.
- `.isalpha()` – neprázdný řetězec jen z písmen.
- `.isdecimal()` – neprázdný řetězec jen z desítkových číslic.
- `<`, `<=` apod. – *lexikografické* porovnání řetězců.



## Zjišťování vlastností řetězců (predikáty)

- `.islower()` – neprázdný řetězec splňující:
  - obsahuje alespoň jedno písmeno,
  - všechna písmena v řetězci jsou malá.
- `.isupper()` – podobně jako výše, všechna velká.
- `.isalpha()` – neprázdný řetězec jen z písmen.
- `.isdecimal()` – neprázdný řetězec jen z desítkových číslic.
- `<`, `<=` apod. – *lexikografické* porovnání řetězců.

## Převody mezi řetězcí a seznamy

- `list(text)` – seznam „znaků“ (jednoznakových řetězců).
- `text.split(sep)` – rozdělení řetězce podle oddělovače.
- `sep.join(list_of_str)` – spojení řetězců oddělovačem.

### Konverze mezi morseovkou a latinkou

- Máme připravené slovníky `LETTER_TO_MORSE` a `MORSE_TO_LETTER`.

## Konverze mezi morseovkou a latinkou

- Máme připravené slovníky LETTER\_TO\_MORSE a MORSE\_TO\_LETTER.

```
def to_morse(text: str) -> str:
    result = ""
    for letter in text.upper():
        if letter.isalpha():
            if result != "":
                result += "|"
            result += LETTER_TO_MORSE[letter]
        elif letter == " ":
            result += " |"
    return result
```

## Konverze mezi morseovkou a latinkou

- Máme připravené slovníky LETTER\_TO\_MORSE a MORSE\_TO\_LETTER.

```
def from_morse(morse: str) -> str:
    result = ""
    for part in morse.split('|'):
        if part == "":
            result += " "
        else:
            result += MORSE_TO_LETTER[part]
    return result
```

## Interakce s prostředím

---

## Obecné schéma základní práce se soubory

- Otevření souboru.
- Práce se souborem: čtení/zápis.
- Zavření souboru.

## Obecné schéma základní práce se soubory

- Otevření souboru.
- Práce se souborem: čtení/zápis.
- Zavření souboru.

## Speciální blok `with`

- Otevře soubor.
- Zajistí automatické zavření souboru při opuštění bloku,
  - jakkoliv k tomu dojde.
  - `return`, `break`, `continue`, výskyt chyby, ...
- (Soubory lze zavírat i „ručně“, ale to zde nechceme dělat.)

```
with open(soubor, režim) as proměnná:  
    # práce se souborem skrze proměnnou
```

```
# na konci bloku se soubor sám zavře
```

- **Jméno (cesta k) souboru:** řetězec (pozor na Windows a '\\').
- **Režim otevření:**
  - čtení ("r") – soubor musí existovat,
  - zápis ("w") – existující soubor je přemazán,
  - exkluzivní zápis ("x") – soubor nesmí existovat,
  - přidání na konec ("a"),
  - a další (mimo záběr předmětu).
- Režimy "w", "x" a "a" soubor vytvoří, pokud neexistuje.
- Implicitní otevření v *textovém módu*,
  - existuje i *binární mód* (mimo záběr předmětu).



## Konec řádku

- Jeden nebo více skutečných znaků dle platformy.
  - *Textový mód*: implicitní konverze vždy na `'\n'`.

## Konec souboru

- POSIXová konvence (*doporučená*): `'\n'` je *konec řádku*.
  - Každý řádek by měl mít konec, tedy i ten *poslední*.
  - Očekává se, že *neprázdný soubor* končí znakem `'\n'`.
- MS DOS/Windows konvence: `'\n'` je *oddělovač řádků*.
  - Poslední řádek tedy nekončí `'\n'`.
  - Nestandardní, má různé problémy.
  - Navíc se používá před `'\n'` ještě `'\r'`.
- Python řeší převážně automaticky.

## Kódování souboru

- Většina moderních OS: implicitně UTF-8 (Unicode).
- MS Windows: implicitně nestandardní CP1250, CP1252 apod.

*Poznámka:* `ib111.py` nastaví UTF-8 jako implicitní.

## Čtení ze souboru

- `f.read(počet)` – přečte zadaný počet znaků, je-li to možné.
- `f.read()` – přečte celý soubor, vrátí řetězec.
- `f.readline()` – přečte celý jeden řádek.
  - Pokud jsme na konci souboru, `read()` a `readline()` vrátí "".
- `f.readlines()` – přečte všechny řádky, vrátí seznam řádků.
- `f.write(text)` – zapíše řetězec do souboru.
  - Neukončuje řádky, je třeba explicitně zapsat `'\n'`.
- Iterace po řádcích `for line in f:`

## Knihovna `sys`

- Různé funkce a objekty pro práci se systémem.
- Zde použijeme pouze pro:
  - získání parametrů z příkazového řádku,
  - standardní vstup, výstup (a chybový výstup).

## Knihovna `sys`

- Různé funkce a objekty pro práci se systémem.
- Zde použijeme pouze pro:
  - získání parametrů z příkazového řádku,
  - standardní vstup, výstup (a chybový výstup).

### `sys.argv`

- Seznam řetězců, obsahuje argumenty, které byly programu předány na **příkazovém řádku**.
- `sys.argv[0]` je samotné jméno programu.
- Podobá se *globální proměnné*.
  - Reálně je `argv` modulová proměnná modulu `sys`.

## Standardní výstup

- Implicitně obrazovka (terminál, konzole).
- Dá se přesměrovat do souboru:
  - v příkazovém řádku `program > soubor` (přepíše soubor)  
nebo `program >> soubor` (přidá na konec souboru).
- Python: `sys.stdout`.
  - Chová se jako soubor, otevřený pro zápis na začátku programu.
  - Neotevíráme ani nezavíráme jej ručně, ale můžeme do něj psát (to ve skutečnosti dělá i procedura `print`).
- Výstup nemusí být okamžitý (využívá vyrovnávací paměť).
  - Proveďte se zápisem znaku konce řádků,
  - nebo explicitně pomocí metody `.flush()`.

## Standardní chybový výstup

- Implicitně opět obrazovka (terminál, konzole).
- Určený pro **logování**, **výpis oznámení o chybách** apod.,
  - tj. to, co není hlavním výstupem programu.
- Přesměrovává se nezávisle na standardním výstupu,
  - může být vidět i při přesměrování standardního výstupu.
- Python: `sys.stderr`.
  - Opět jako soubor otevřený pro zápis na začátku programu.

## Standardní vstup

- Implicitně klávesnice.
- Dá se přesměrovat ze souboru:
  - v příkazovém řádku `program < soubor`.
- Python: `sys.stdin`,
  - opět jako soubor otevřený pro čtení na začátku programu.

## Všimněte si

- `sys.stdin`, `sys.stdout`, `sys.stderr`  
se opět podobají *globálním proměnným*.



### Výpočet výšky z délky volného pádu

- Chceme interaktivní program, který se zeptá na délku pádu a odpoví výškou.
- Budeme potřebovat načíst *číslo*,
  - ale v `ib111.py` nemáme konverze z řetězců na čísla a naopak,
  - abychom si je zkusili napsat *sami*.

## Co je špatně?

```
def main() -> None:
    sys.stdout.write("Zadej dobu v sekundách: ")
    sys.stdout.flush()
    seconds = to_int(sys.stdin.readline().rstrip())
    # ...

def to_int(digits: str) -> int:
    assert digits.isdecimal()
    # ...
```

## Co je špatně?

```
def main() -> None:
    sys.stdout.write("Zadej dobu v sekundách: ")
    sys.stdout.flush()
    seconds = to_int(sys.stdin.readline().rstrip())
    # ...
```

```
def to_int(digits: str) -> int:
    assert digits.isdecimal()
    # ...
```

- Nepoužívejte `assert` ke kontrole *uživatelského vstupu*.
- S chybami je třeba se vypořádat jinak:
  - např. vracet si z pomocné funkce `None` a uživateli pak vypsát srozumitelnou chybovou hlášku.

## Rekurze pro rozbor textu

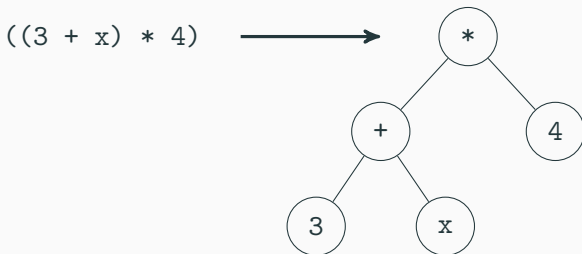
---

## Syntaktická analýza, rozbor textu (*parsing*)

- Analýza textu, jejímž cílem je rozpoznat jeho strukturu.
- Příklad: rozpoznání aritmetických výrazů.
- Různé metody, se kterými se seznámíte v průběhu studia.
- Jedna z metod – **metoda rekurzivního sestupu**,
  - *recursive descent parsing*.
- Zde si ji ukážeme jen na velmi jednoduchém příkladu:
  - další ukázka reálného použití rekurze.

## Příklad: Aritmetické výrazy

- **Vstup:** textový řetězec s aritmetickým výrazem,
  - uvažujeme pouze operátory + a \*,  
(desítkově zapsaná) čísla a proměnné,
  - pro začátek uvažujeme pouze *plně uzávorkované výrazy*.
- **Výstup:** stromová reprezentace výrazu.



```
class Op:
    def __init__(self, op: str,
                  left: 'Expression',
                  right: 'Expression'):
        self.op = op
        self.left = left
        self.right = right
```

```
Expression = int | str | Op
```

- Máme připravenou proceduru `draw_graphviz`, která nám strom výrazu vykreslí ve formátu Graphviz.
  - Pro zobrazení použijte webovou verzi: <http://webgraphviz.com> nebo si nainstaluje nástroj `xdot`.

**První krok** – rozložení textu na symboly (*tokeny*).

- Tzv. *lexikální analýza*.
- Zatím žádná rekurze, čistě řetězcové operace.
- Druhy symbolů (*tokenů*):
  - levá závorka, pravá závorka, plus, krát,
  - číslo (v desítkovém zápisu),
  - proměnná (posloupnost písmen a číslic, začíná písmenem),
  - přidáme speciální token „konec vstupu“ – prázdný řetězec (bude užitečný za chvíli).
- Chceme ignorovat mezery.



**První krok** – rozložení textu na symboly (*tokeny*).

- Tzv. *lexikální analýza*.
- Zatím žádná rekurze, čistě řetězcové operace.
- Druhy symbolů (*tokenů*):
  - levá závorka, pravá závorka, plus, krát,
  - číslo (v desítkovém zápisu),
  - proměnná (posloupnost písmen a číslic, začíná písmenem),
  - přidáme speciální token „konec vstupu“ – prázdný řetězec (bude užitečný za chvíli).
- Chceme ignorovat mezery.
- Jak se vypořádat s neplatným vstupem?
  - Vstupní podmínka 1 – pouze platný vstup: **assert**.
  - Vstupní podmínka 2 – cokoli:
    - v případě chyby vrátíme **None** nebo jinou speciální hodnotu.

**Druhý krok** – vytvoření stromu.

- Chceme použít rekurzi – ale jak?

**Druhý krok** – vytvoření stromu.

- Chceme použít rekurzi – ale jak?
  - Je třeba si rozmyslet, jak jsou definované aritmetické výrazy.

**Druhý krok** – vytvoření stromu.

- Chceme použít rekurzi – ale jak?
  - Je třeba si rozmyslet, jak jsou definované aritmetické výrazy.

**Definice aritmetického výrazu** (induktivní)

- Desítkový zápis čísla je aritmetický výraz.
- Jméno proměnné je aritmetický výraz.
- Pokud  $X$  a  $Y$  jsou aritmetické výrazy, pak také
  - $( X + Y )$  je aritmetický výraz a
  - $( X * Y )$  je aritmetický výraz.
- Rekursivní analýza bude kopírovat strukturu indukativní definice.

## Třída pro analyzátor (*parser*)

- Budeme si udržovat seznam tokenů a aktuální pozici.
- Pomocné metody:
  - `peek` – vrátí token na aktuální pozici,
  - `consume` – posune pozici o jednu dále,
  - `next` – vrátí token na aktuální pozici (jako `peek`), ale zároveň ji posune (jako `consume`).

```
class Parser:  
    def __init__(self, expr: str):  
        self.tokens = to_tokens(expr)  
        self.index = 0
```

```
def expression(self) -> 'Expression':
    token = self.next()

    if token == '(':
        left = self.expression()
        op = self.next()
        right = self.expression()
        self.consume() # ')'
        return Op(op, left, right)

    if token.isdecimal():
        return to_int(token)

    # it has to be a variable name
    return token
```

## Hlavní funkce

```
def parse_expression(expr: str) -> 'Expression':  
    return Parser(expr).expression()
```

## Hlavní funkce

```
def parse_expression(expr: str) -> 'Expression':  
    return Parser(expr).expression()
```

## Co s chybným vstupem?

- Verze se vstupní podmínkou:
  - možná `assert` na vhodná místa,
  - pro jistotu, pro vlastní kontrolu.
- Verze s volným vstupem:
  - testovat, zda se objevují očekávané tokeny,
  - vracet a ověřovat `None`,
  - trochu pracné, ale zvládnutelné,
  - lepší řešení by reportovalo, kde došlo k chybě a jaké (např. místo `None` vracet objekt s informací o chybě).



## Co to zkusit bez závorek?

- Povolíme i výrazy tvaru  $3 + 4 + 5$  nebo  $3 + 4 * 5$ .
- Co to vlastně znamená?

## Co to zkusit bez závorek?

- Povolíme i výrazy tvaru  $3 + 4 + 5$  nebo  $3 + 4 * 5$ .
- Co to vlastně znamená?
  - Potřebujeme si ujasnit **prioritu operátorů** a **asociativitu**.

## Co to zkusit bez závorek?

- Povolíme i výrazy tvaru  $3 + 4 + 5$  nebo  $3 + 4 * 5$ .
- Co to vlastně znamená?
  - Potřebujeme si ujasnit **prioritu operátorů** a **asociativitu**.

## Priorita operátorů

- $*$  má větší prioritu než  $+$ ,
- tj.  $3 + 4 * 5$  znamená  $(3 + (4 * 5))$ .

## Asociativita

- Oba operátory asociují *zleva*,
- tj.  $3 + 4 + 5$  znamená  $((3 + 4) + 5)$ .

## Definice výrazů bez závorek

- **Výraz** je posloupnost  $sčítanec + sčítanec + \dots + sčítanec$ ,
  - tj. jeden nebo více *sčítanců* oddělených symbolem  $+$ .
- **Sčítanec** je posloupnost  $činitel * činitel * \dots * činitel$ ,
  - tj. jeden nebo více *činitelů* oddělených symbolem  $*$ .
- **Činitel** je
  - desítkový zápis čísla,
  - jméno proměnné,
  - ( *výraz* ).
- Tuto definici opět využijeme při budování analyzátoru.
  - Anglické názvy: *expression*, *term*, *factor*.
- Všimněte si: **nepřímá indukce**  $\rightarrow$  **nepřímá rekurze**.

**Výraz** = *sčítanec* + *sčítanec* + ... + *sčítanec*.

```
def expression(self) -> 'Expression':  
    current = self.term()  
    while self.peek() == '+':  
        op = self.next()  
        right = self.term()  
  
        current = Op(op, current, right)  
  
    return current
```

**Sčítanec** = *činitel* \* *činitel* \* ... \* *činitel*

```
def term(self) -> 'Expression':  
    current = self.factor()  
    while self.peek() == '*':  
        op = self.next()  
        right = self.factor()  
  
        current = Op(op, current, right)  
  
    return current
```

**Činitel** = číslo nebo proměnná nebo ( *výraz* )

```
def factor(self) -> 'Expression':  
    token = self.next()  
    if token == '(':  
        expr = self.expression()  
        self.consume() # ')'  
        return expr  
  
    if token.isdecimal():  
        return to_int(token)  
  
    # it has to be a variable name  
    return token
```

**Všimli jste si něčeho?**



## Všimli jste si něčeho?

- Funkce `expression` a `term` jsou skoro stejné.
- Předchozí řešení není DRY.

## Lepší řešení

- Jedna funkce místo `expression` a `term`:
  - úroveň priority operátorů jako parametr,
  - úroveň 0 je sčítání,
  - úroveň 1 je násobení,
  - úroveň 2 je číslo nebo proměnná nebo ozávkovaný výraz *úrovně 0*.
- Snadno rozšiřitelné o další operátory.
  - (Pro úplnou obecnost: asociativita zprava, unární operátory, ...)

```
def expression(self, level: int) -> 'Expression':  
    if level == len(OP_LEVELS):  
        return self.simple_expr()  
  
    current = self.expression(level + 1)  
    while self.peek() == OP_LEVELS[level]:  
        op = self.next()  
        right = self.expression(level + 1)  
  
        current = Op(op, current, right)  
  
    return current
```

- `simple_expr` je bývalý `factor`.
- Opět můžeme přidat ověřování chyb.

## **Analyzátor jednoduchých aritmetických výrazů**

- využívá metodu rekurzivního sestupu
- pouze pomocí operací definovaných v našem omezeném jazyce

## Analyzátor jednoduchých aritmetických výrazů

- využívá metodu rekurzivního sestupu
- pouze pomocí operací definovaných v našem omezeném jazyce

## Možná rozšíření (pro pokročilé)

- Více operátorů:
  - unární operátory,
  - zprava asociativní operátory (např. \*\*).
- Lepší reportování chyb:
  - výpis na `sys.stderr`; ideálně až v hlavní funkci.
  - Nápad: držet si informaci o chybě v atributu třídy.
  - Jiný nápad: místo `None` si vracet informaci o chybě (např. zabalenou v objektu pomocné třídy `Error`).

## Analyzátor jednoduchých aritmetických výrazů

- využívá metodu rekurzivního sestupu
- pouze pomocí operací definovaných v našem omezeném jazyce

## Možná rozšíření (pro pokročilé)

- Více operátorů:
  - unární operátory,
  - zprava asociativní operátory (např. \*\*).
- Lepší reportování chyb:
  - výpis na `sys.stderr`; ideálně až v hlavní funkci.
  - Nápad: držet si informaci o chybě v atributu třídy.
  - Jiný nápad: místo `None` si vracet informaci o chybě (např. zabalenou v objektu pomocné třídy `Error`).

**Příště:** vlastní interpret jednoduchého jazyka.